UNIVERSITÀ DI PISA

DIPARTIMENTO DI INGEGNERIA DELL'INFORMAZIONE

Laurea Triennale in Ingegneria Informatica

Ottimizzazione del 3D Bin Packing Problem per la logistica dell'ultimo miglio: Integrazione di vincoli di accessibilità e stabilità dinamica.

Relatore:

Prof: Mario Giovanni Cosimo Antonio Cimino

Candidato:

Lorenzo Salvatelli

ANNO ACCADEMICO 2025/2026

Indice

1 Introduzione **7**
1.1 Contesto 7
1.2 Definizione del problema 7
1.3 Obiettivi 8

2 Il Problema del Bin Packing **9**
2.1 Definizione matematica e modellizzazione 9
2.1.1 Il modello matematico 9
2.2 Complessità computazionale 10
2.2.1 L'impossibilità pratica del calcolo esatto 11
2.3 Gli algoritmi greedy 11

3 Ambiente di lavoro **13**
3.1 Linguaggio e ambiente di sviluppo 13
3.2 La libreria py3dbl 13
3.2.1 Struttura dei moduli e classi principali 14
3.2.2 Strategie di packing 14
3.2.3 Il sistema di generazione dei colli 14
3.2.4 Il sistema di gestione dei vincoli 15
3.2.5 Visualizzazione dei risultati 15

4 Estensioni algoritmiche e analisi sperimentale **17**
4.1 Ridefinizione dei criteri di ordinamento: da peso a priorità 17
4.2 Implementazione del vincolo del corridoio centrale 18
4.3 Algoritmo di calcolo delle mosse operative 19
4.4 Persistenza dei dati e generazione dei report (Excel) 23
4.5 Generazione dei grafici di sintesi in HTML 24
4.6 Studio dell'impatto del corridoio e scalabilità 24
4.7 Simulazione 3D e animazione dello scarico 25

5 Analisi dei risultati e conclusioni **29**
5.1 Configurazione dello Stress Test 29
5.2 Analisi delle metriche sperimentali 29
5.2.1 Impatto sulla capacità di carico (Saturazione) 29

5.2.2 Impatto sull'efficienza operativa 30
5.2.3 Analisi della stabilità (distanza dal Centro di Gravità) 31
5.3 Conclusioni e identificazione del punto di ottimo 31
5.4 Obiettivi futuri 33

Elenco delle figure

4.1 Caricamento senza corridoio 27
4.2 Caricamento con corridoio 28

5.1 Saturazione volumetrica. 30
5.2 Efficienza operativa. 30
5.3 Distanza del CoG dal punto di ideale. 31
5.4 Sovrapposizione fra i grafici. 32

Capitolo 1

Introduzione

1.1 Contesto

Nell'attuale contesto economico, caratterizzato dal commercio elettronico e da catene di approvvigionamento sempre più rapide e reattive, la logistica dei trasporti ha assunto un ruolo di importanza strategica.

L'efficienza dei vettori logistici non si misura più esclusivamente attraverso la riduzione dei consumi di carburante o attraverso l'abbattimento delle emissioni di CO_2, ma dipende in modo crescente dalla capacità di ottimizzare i processi operativi in ogni loro fase.

Con specifico riguardo all'ultima fase del trasporto (*last mile*), la ricerca accademica ed industriale ha tradizionalmente affrontato il problema del caricamento e scaricamento dei pacchi ponendo come obiettivo primario la massimizzazione della saturazione volumetrica del vano di carico. Tuttavia, l'esperienza operativa sul campo evidenzia come tale approccio non sia più soddisfacente in quanto un carico, se non opportunamente organizzato, possa costituire un collo di bottiglia durante la fase di consegna, con conseguenti inefficienze.

1.2 Definizione del problema

L'ottimizzazione del carico logistico è nota in letteratura come *3D Bin Packing Problem (3D-BPP)* e la sua complessità cresce esponenzialmente al crescere del numero di colli e della varietà delle loro dimensioni. Tuttavia, nel contesto reale del corriere espresso, la sfida non si limita al mero incastro geometrico dei volumi.

Esistono infatti vincoli stringenti che condizionano la fattibilità di un piano di carico:

- Vincoli fisici: in fase di caricamento è imprescindibile tener fede alla fisica degli oggetti. Ad esempio, due colli non possono compenetrarsi, né possono essere impilati se il superiore poggia su una superficie insufficiente all'equilibrio;

ancora, i pacchi devono rientrare completamente nel volume del vano di carico, permettendo anche la chiusura del portellone posteriore.

- Vincoli di stabilità e sicurezza: la distribuzione dei pesi deve garantire che il centro di gravità del veicolo rimanga entro tolleranze specifiche, per evitare sbilanciamenti pericolosi durante la marcia; inoltre, ogni collo deve godere di un supporto fisico adeguato ad evitare schiacciamenti o cadute.

- Vincoli di accessibilità e ordine di scarico: in una missione di consegna con più fermate (multi-stop) i pacchi devono essere caricati seguendo un ordine preciso, basato sulla successiva priorità di estrazione (logica Last-In-First-Out o LIFO); un ordine di caricamento basato esclusivamente sul volume del pacco tende a "seppellire" i colli in fondo al vano di carico, costringendo l'operatore ad attuare reiterati spostamenti di colli per liberare la merce da consegnare.

- Configurazione del vano di carico: l'introduzione di elementi strutturali volti a facilitare il lavoro umano, come un corridoio centrale d'ispezione [1], sottrae volume utile al carico; in tal caso, si genera un *trade-off* per cui, la maggiore saturazione dello spazio disponibile comporta una riduzione dell'efficienza operativa nelle fasi di scarico intermedio.

1.3 Obiettivi

Questo studio si pone dunque l'obiettivo di valutare l'introduzione di vincoli inerenti alla facilità di scarico delle merci rispetto alla tradizionale impostazione del problema del bin packing, mirando a massimizzare l'efficienza operativa del carico.

Contestualmente, verranno monitorati sia l'utilizzo volumetrico di ogni caricamento, sia il calcolo del centro di gravità del carico.

Il risultato atteso è un algoritmo che soddisfi tali necessità.

Capitolo 2

Il Problema del Bin Packing

2.1 Definizione matematica e modellizzazione

Il problema del 3D Bin Packing (3D-BPP) appartiene alla famiglia dei problemi di ottimizzazione combinatoria. Nella sua formulazione classica, l'obiettivo è stivare un insieme di oggetti tridimensionali all'interno del minor numero possibile di contenitori (*bin*).

Nel contesto specifico di questa tesi, dove il contenitore è unico (il vano di carico del mezzo di trasporto) e l'obiettivo è massimizzare il volume sfruttato rispettando la capacità di peso, il problema viene più propriamente definito come 3D Container Loading Problem (3D-CLP).

2.1.1 Il modello matematico

Possiamo formalizzare il problema attraverso un modello di programmazione lineare intera.
Sia dato un contenitore (Bin) di dimensioni note:

W (larghezza), H (altezza) e D (profondità).

Lo spazio cartesiano interno è delimitato dal volume

$$V = W \cdot H \cdot D.$$

Sia

$$N = \{1,\ 2,\ \ldots,\ n\}$$

l'insieme degli n oggetti da caricare.

Ogni oggetto $i \in N$ è caratterizzato da:

- Dimensioni: w_i (larghezza),h_i (altezza), d_i (profondità).

- Volume: $v_i = w_i \cdot h_i \cdot d_i$.
- Peso: p_i.

Definiamo le seguenti variabili decisionali:

- $u_i \in 0, 1$: variabile binaria che assume valore 1 se l'oggetto i viene inserito nel bin, 0 altrimenti.
- (x_i, y_i, z_i): coordinate reali nello spazio 3D che identificano il vertice anteriore-inferiore-sinistro dell'oggetto i all'interno del bin (origine nel vertice anteriore-inferiore-sinistro del cassone).

L'obiettivo primario del problema consiste nel massimizzare il volume totale degli oggetti caricati:

$$\sum_{i=1}^{n} v_i \ \cdot u_i$$

Soggetto ai seguenti vincoli fondamentali:

- Vincolo di contenimento (i pacchi non devono uscire dal volume di carico del mezzo):
  Per ogni oggetto i per cui $u_i = 1$:

$$\begin{gathered} 0 \leq x_i \leq W - w_i \\ 0 \leq y_i \leq H - h_i \\ 0 \leq z_i \leq D - d_i \end{gathered}$$

- Vincolo di non-compenetrazione (due pacchi non possono occupare lo stesso spazio): Per ogni coppia di oggetti distinti i,j caricati ($u_i = 1, u_j = 1$), questi non devono compenetrarsi.
  Matematicamente, deve essere soddisfatta almeno una delle seguenti sei condizioni di disuguaglianza spaziale:

$$\begin{gathered} (x_i + w_i \leq x_j) \vee (x_j + w_j \leq x_i) \\ \vee \\ (y_i + h_i \leq y_j) \vee (y_j + h_j \leq y_i) \\ \vee \\ (z_i + d_i \leq z_j) \vee (z_j + d_j \leq z_i) \end{gathered}$$

2.2 Complessità computazionale

Dal punto di vista della teoria della complessità computazionale, il problema del Bin Packing (anche nella sua versione più semplice unidimensionale) è classificato come NP-hard *(Non-deterministic Polynomial-time hard)*.

Ciò significa che non si conoscono (e con ogni probabilità non esistono) algoritmi in grado di trovare la soluzione ottima globale, in un tempo che cresca in modo polinomiale, rispetto al numero di oggetti n. Al crescere di n, il numero di combinazioni possibili per disporre gli oggetti e le loro eventuali rotazioni aumenta in modo esponenziale.

2.2.1 L'impossibilità pratica del calcolo esatto

I metodi risolutivi esatti, come gli algoritmi di *Branch and Bound* o la programmazione lineare intera pura, tentano di esplorare l'intero albero delle soluzioni. Sebbene garantiscano il raggiungimento dell'ottimo matematico, richiedono risorse computazionali e tempi di calcolo proibitivi non appena il numero di pacchi supera poche decine di unità.

Nelle applicazioni reali della logistica dell'ultimo miglio, dove i corrieri devono calcolare in pochi secondi prima della partenza i piani di carico per decine o centinaia di pacchi, l'utilizzo di algoritmi esatti è matematicamente e operativamente impossibile. Si rende quindi necessario l'abbandono della ricerca dell'ottimo perfetto a favore di soluzioni sub-ottime ma calcolabili rapidamente.

2.3 Gli algoritmi greedy

Per far fronte all'eccessiva complessità computazionale del 3D-BPP, la letteratura scientifica e la pratica ingegneristica fanno largo uso di euristiche, tra cui spiccano per diffusione gli algoritmi greedy.

Un algoritmo greedy opera compiendo, ad ogni singolo passo, la scelta che risulta localmente più ottima o conveniente in quel preciso momento, senza mai rimettere in discussione le decisioni passate (*assenza di backtracking*) e senza calcolare le conseguenze sulle scelte future.

Nel contesto del packing tridimensionale, un tipico approccio greedy funziona secondo questa sequenza:

1. Ordina gli oggetti secondo un criterio prefissato (es. dal più grande al più piccolo, o per priorità).
2. Prende il primo oggetto della lista.
3. Scansiona lo spazio vuoto del bin e inserisce l'oggetto nella prima posizione valida trovata che rispetti i vincoli geometrici.
4. Ripete per tutti gli oggetti fino a esaurimento spazio o elementi.

Potenzialità degli approcci Greedy:

- Efficienza temporale e scalabilità: richiedono tempi di calcolo estremamente ridotti, solitamente con complessità nell'ordine di $O\left(n \cdot log_n\right)$ o $O\left(n^2\right)$.
  Ciò permette di elaborare soluzioni per centinaia di pacchi in frazioni di secondo.

- Semplicità implementativa: sono facili da tradurre in codice e non richiedono complessi motori di ottimizzazione esterni.

- Adattabilità: è relativamente semplice inserire nuovi vincoli all'interno del ciclo di controllo.

Limiti e criticità:

- Ottimi locali: l'assenza di una visione globale porta l'algoritmo a "inscatolare" scelte miopi; spesso il riempimento iniziale preclude lo spazio a oggetti successivi più grandi, lasciando ampi vuoti inutilizzabili.

- Sensibilità all'ordinamento: il risultato finale dipende largamente da come la lista iniziale viene ordinata. Piccole variazioni nell'ordine di input possono portare a risultati molto differenti e imprevedibili.

- Inefficienza nello spazio 3D: negli spazi tridimensionali, le euristiche greedy pure (come il posizionamento ad angolo fisso) tendono a creare "scalini" e superfici frastagliate che rendono impossibile l'incastro dei carichi successivi, abbassando notevolmente la percentuale di saturazione del volume del mezzo di trasporto.

Al fine di mitigare questi limiti strutturali degli algoritmi greedy puri, all'interno della libreria py3dbl utilizzata per questa tesi, si è preferito evolvere l'approccio classico verso strategie più evolute come il *Multi-Anchor* (oggetto di trattazione nei capitoli applicativi).

Capitolo 3

Ambiente di lavoro

3.1 Linguaggio e ambiente di sviluppo

La progettazione, l'implementazione e la validazione del sistema di ottimizzazione del carico sono state interamente realizzate sfruttando il linguaggio di programmazione Python. Questa scelta è stata dettata dall'estrema flessibilità del linguaggio e dall'ampio spettro di librerie disponibili per l'elaborazione scientifica e la visualizzazione dei dati.

Per la definizione di ogni dimensione, peso e coordinata spaziale, si è fatto sistematico ricorso al tipo `Decimal` appartenente alla libreria nativa di Python; ciò al fine di ovviare ai noti problemi di imprecisione numerica tipici dell'aritmetica in virgola mobile (*floating-point*); infatti, in un problema geometrico tridimensionale la suddetta notazione potrebbe causare compenetrazioni involontarie o errori nel calcolo dei volumi.

Il sistema prevede inoltre una forte componente visiva e analitica; in particolare, è stata impiegata la libreria Plotly per il rendering tridimensionale del vano di carico del furgone e per la generazione di grafici interattivi relativi alle metriche di efficienza e centro di gravità.

L'orchestrazione dei test in serie per simulare e comparare vari scenari è stata invece gestita per mezzo di script di automazione Bash.

3.2 La libreria py3dbl

Il cuore tecnologico di questo lavoro poggia sulla libreria py3dbl (Python 3D Bin Packing Library).

3.2.1 Struttura dei moduli e classi principali

La libreria si presenta suddivisa in vari moduli altamente specializzati che cooperano per la risoluzione del problema di carico da un punto di vista fisico:

- `Bin.py`: contiene le classi BinModel e Bin, le quali descrivono i parametri fisici e la capacità massima del contenitore, implementando l'effettiva logica di inserimento degli oggetti fisici e il calcolo della massa totale.
- `Item.py`: rappresenta il singolo pacco da stivare, provvisto di dimensioni spaziali tridimensionali, peso e un livello di priorità di consegna dinamico.
- `Space.py`: introduce i concetti matematici fondamentali di spazio tramite le strutture vettoriali Vector3 e i volumi cubici Volume.

3.2.2 Strategie di packing

L'assegnazione e il posizionamento degli oggetti all'interno del mezzo di trasporto sono governati dal modulo `Packer.py`, che prevede l'applicazione di differenti logiche algoritmiche:

1. Strategia Greedy (Left-Bottom-Back): questo metodo euristico adotta un approccio "goloso", tentando di posizionare ogni oggetto dall'angolo inferiore-posteriore-sinistro e accettando la prima collocazione geometricamente valida. Tuttavia, poiché l'algoritmo tende a concentrare gli oggetti nello stesso angolo di partenza, questo metodo rischia di sbilanciare gravemente il peso del carico complessivo, di non offrire sufficiente diversificazione nelle scelte, di ostacolare il lavoro di scarico da parte di un operatore.
2. Strategia Multi-Anchor: Per porre rimedio ai limiti della logica greedy (LBB), è stata implementata una strategia avanzata basata su ancoraggi multipli. Tale approccio non si ferma alla prima posizione trovata, bensì calcola un insieme di coordinate (X, Z) candidate a partire dagli angoli del pavimento, dal centro geometrico del furgone e dalle zone strettamente adiacenti agli oggetti già posizionati. Ognuna di queste combinazioni viene soppesata mediante una funzione di punteggio (scoring function), la quale privilegia sia l'altezza di posizionamento minima, sia la compattezza totale, e assicura un bilanciamento più omogeneo della distribuzione spaziale dei carichi.

3.2.3 Il sistema di generazione dei colli

Al fine di una corretta simulazione di caricamento, è stato predisposto il file `item_generator.py`, il quale è capace di generare una lista di colli con caratteristiche diverse.

La randomizzazione dei pacchi è di vitale importanza per poter simulare uno scenario verosimile, potendo così attuare uno studio robusto e completo.

3.2.4 Il sistema di gestione dei vincoli

Uno dei contributi di maggiore rilievo all'interno dell'estensione della libreria risiede nell'implementazione flessibile dei vincoli fisici e operativi. Nel file Constraints.py è stata definita una classe Constraint che impiega la sintassi dei decoratori Python (`@constraint`) per assegnare un peso e una logica di valutazione ad ogni singola regola. L'ordinamento dei vincoli in base al loro peso computazionale assicura che le condizioni matematicamente più onerose vengano verificate per ultime, ottimizzando sensibilmente i tempi di esecuzione dell'algoritmo.

Tra i vincoli di base (definiti dall'array `BASE_CONSTRAINTS`) applicati sistematicamente figurano:

- `weight_within_limit`: impedisce di eccedere la portata massima in termini di peso del furgone;
- `fits_inside_bin`: controlla che le coordinate fisiche occupate dall'oggetto non sforino le dimensioni delle pareti del mezzo;
- `no_overlap`: assicura che due oggetti differenti non si compenetrino all'interno dello spazio tridimensionale.
- `is_supported`: esamina matematicamente se la superficie inferiore di un pacco poggi con un rapporto percentuale sufficiente sopra altri pacchi o direttamente sul pianale del veicolo.

L'architettura estensibile permette inoltre di applicare regole aggiuntive come:

- `maintain_center_of_gravity`: si tratta di un vincolo che impedisce il posizionamento di un collo in una posizione in cui determinerebbe lo spostamento del centro di gravità del carico al sopra di una certa soglia di tolleranza.
- `central_corridor_accessibility`: il vincolo di accessibilità oggetto di studio in questa tesi, che ha la funzione di mantenere sgombro un corridoio d'accesso per l'operatore, al fine di aumentare la superficie di accesso ai pacchi da parte dell'operatore.
  Questa scelta di caricamento è già largamente usata nella pratica delle compagnie di consegna e riduce drasticamente il tempo che l'operatore deve impiegare in ogni fermata nel giro di consegne.

3.2.5 Visualizzazione dei risultati

Il `test_slider.py` costituisce il modulo fondamentale per la visualizzazione dei risultati del caricamento.

Attraverso questo codice di testing si può decidere il numero dei colli da caricare e la larghezza del corridoio di accesso. Un corridoio di larghezza pari a zero

corrisponde al caricamento precedentemente sviluppato nella tesi del Dott. Francesco Chiera, senza il mantenimento del centro di gravità.

Terminato il test, viene creata una pagina HTML nella quale è possibile osservare il cassone del camion, caricato nel momento della partenza. Inoltre, per ogni test eseguito, il file crea o aggiorna un file .csv che tiene traccia dei risultati calcolati in tale simulazione.

Infine, lo script crea una pagina HTML all'interno della quale vengono presentati tre grafici che misurano l'impatto del corridoio centrale sulla quantità di oggetti caricati, sull'efficienza operativa di scarico e sulla distanza del centro di gravità del carico dalle coordinate di idealità.

Capitolo 4

Estensioni algoritmiche e analisi sperimentale

In questo capitolo viene descritto il contributo apportato da questa tesi alla libreria py3dbl al fine di adattarla alle reali esigenze della logistica dell'ultimo miglio. L'obiettivo precipuo della sperimentazione è duplice: da un lato, rendere lo scarico più semplice e veloce, dall'altro, studiare matematicamente il compromesso (*trade-off*) tra la perdita di volume utile e il guadagno in termini di accessibilità al variare delle dimensioni del carico.

4.1 Ridefinizione dei criteri di ordinamento: da peso a priorità

Nella versione originale della libreria e nei classici approcci al Bin Packing, il vettore contenente gli oggetti (items) veniva ordinato principalmente in base a criteri geometrici o fisici, come il volume o il peso decrescente. Questo approccio, pur massimizzando la stabilità strutturale del carico, ignora completamente la sequenza logica delle consegne.

Per risolvere questo problema la logica di inserimento è stata modificata, introducendo un ordinamento basato sulla priorità di consegna:

- Logica precedente: gli oggetti più voluminosi venivano inseriti per primi sul fondo del vano di carico.

```
 items_to_pack.sort(
 key=lambda item: item.volume.volume(), reverse=
bigger_first
 )

```

- Nuova logica: gli oggetti vengono ordinati in base alla priorità del punto di scarico (assegnata dinamicamente).

```
    items_to_pack.sort(
    key=lambda item: (item.priority, item.volume.volume())
    )

```

In questa nuova logica, l'algoritmo di packing processa per ultimi gli oggetti destinati alle prime consegne; di conseguenza, dato che l'algoritmo riempie il vano dal fondo verso il portellone posteriore, questi oggetti finiscono per trovarsi nelle posizioni più esterne e raggiungibili del furgone, riducendo drasticamente il tempo di sosta del mezzo.

4.2 Implementazione del vincolo del corridoio centrale

Il secondo intervento ha riguardato l'inserimento di barriere fisiche all'interno del vano di carico. Infatti, un furgone stipato al 100% della sua capacità volumetrica costringe l'operatore a reiterate movimentazioni di pacchi intermedi per raggiungere un oggetto sepolto sul fondo.

Per ovviare a questo problema, è stato introdotto nel file `Constraints.py` il vincolo `central_corridor_accessibility`.

```
@constraint(weight=21)
def central_corridor_accessibility(bin: Bin, item: Item,
                                   corridor_width_percent: float = 0.2,
                                   corridor_depth_percent: float = 0.85):
    """
    Centered aisle, created in order to reduce items movements and to make worker access easier

    :param corridor_width_percent: Percentage of total width dedicated to the aisle (es. 0.2 = 20%)
    :param corridor_depth_percent: Percentage of total depth dedicated to the aisle (es. 0.85 = 85%)
    """

    # Calculating aisle x limits
    half_width_pct = Decimal(str(corridor_width_percent)) / 2
    bin_center_x = bin.width / 2

```

```
    # Calculating aisle z limits
    corridor_z_max = bin.depth * (1 - Decimal(str(
corridor_depth_percent)))

    corridor_x_min = bin_center_x - (bin.width *
half_width_pct)
    corridor_x_max = bin_center_x + (bin.width *
half_width_pct)

    # Item position
    item_x_min = item.position.x
    item_x_max = item.position.x + item.width
    item_z_min = item.position.z + item.depth

    # Verifying intersections
    intersects_x = not (item_x_max <= corridor_x_min or
item_x_min >= corridor_x_max)
    intersects_z = (item_z_min >= corridor_z_max)

    if (intersects_x and intersects_z):
        return False # Collision detected

    return True
```

Questo algoritmo:

1. Riceve in input una percentuale (es. 20%), la quale rappresenta la frazione della larghezza totale del furgone da mantenere sgombra come corridoio di accesso. Agisce come un "muro virtuale": se l'ingombro di un pacco tenta di invadere le coordinate riservate al corridoio, il vincolo fallisce e l'algoritmo cerca una nuova disposizione per il pacco.

2. Ancora, riceve in input una seconda percentuale (es. 85%) che rappresenta la frazione della profondità totale del furgone da mantenere sgombra, in quanto il corridoio non raggiunge il fondo del furgone ma si arresta di fronte alla prima linea di carico.

3. Calcola dinamicamente l'area geometrica del corridoio all'interno del vano di carico.

L'algoritmo garantisce all'operatore un percorso calpestabile continuo dal portellone posteriore fino alla profondità desiderata del vano di carico.

4.3 Algoritmo di calcolo delle mosse operative

Per apprezzare il miglioramento introdotto dal corridoio di accesso non era sufficiente osservare la simulazione offerta dal `test_slider.py`, perché essa non quantifica

le mosse. È stato quindi sviluppato un algoritmo dedicato al calcolo delle mosse necessarie allo scarico.

```
1 def calculate_moves_v5(bin_obj: Bin, has_corridor: bool):
2     items = bin_obj.items
3     bw, bd = float(bin_obj.width), float(bin_obj.depth)
4     c_half = (bw * CORRIDOR_WIDTH_PCT) / 2
5     c_min, c_max = (bw / 2) - c_half, (bw / 2) + c_half
6     priorities = sorted(list(set(int(i.priority) for i in
    items)), reverse=True)
7     unloading_results = {}
8
9     def get_all_blockers(target, current_context, memo,
    visited):
10        if target.name in memo: return memo[target.name]
11        if target.name in visited: return set()
12        visited.add(target.name)
13        tx, ty, tz = float(target.position.x), float(target.
    position.y), float(target.position.z)
14        tw, th, td = float(target.width), float(target.height)
    , float(target.depth)
15
16        top_blockers = set()
17        for other in current_context:
18            if (other.name == target.name or other.priority ==
     target.priority): continue
19            ox, oy, oz = float(other.position.x), float(other.
    position.y), float(other.position.z)
20            ow, oh, od = float(other.width), float(other.
    height), float(other.depth)
21            if oy >= (ty + th - 0.005):
22                if not (tx + tw <= ox + 0.005 or tx >= ox + ow
     - 0.005 or tz + td <= oz + 0.005 or tz >= oz + od - 0.005)
    :
23                    top_blockers.add(other.name)
24                    top_blockers.update(get_all_blockers(other
    , current_context, memo, visited))
25
26        rear_path = set()
27        for other in current_context:
28            if (other.name == target.name or other.priority ==
     target.priority): continue
29            ox, oy, oz = float(other.position.x), float(other.
    position.y), float(other.position.z)
30            ow, oh, od = float(other.width), float(other.
    height), float(other.depth)
31            if oz >= (tz + td - 0.005):
32                if not (tx + tw <= ox + 0.005 or tx >= ox + ow
     - 0.005 or ty + th <= oy + 0.005 or ty >= oy + oh - 0.005)
    :
33                    rear_path.add(other.name)
```

```
                    rear_path.update(get_all_blockers(other, current_context, memo, visited))

        corridor_path = set()
        can_use_corr = False
        if has_corridor:
            is_left = (tx + tw/2) < (bw/2)
            if (is_left and abs(tx + tw - c_min) < 0.03) or (not is_left and abs(tx - c_max) < 0.03): can_use_corr = True
            else:
                for other in current_context:
                    if (other.name == target.name or other.priority == target.priority): continue
                    ox, oy, oz = float(other.position.x), float(other.position.y), float(other.position.z)
                    ow, oh, od = float(other.width), float(other.height), float(other.depth)
                    overlap_yz = not (ty + th <= oy + 0.005 or ty >= oy + oh - 0.005 or tz + td <= oz + 0.005 or tz >= oz + od - 0.005)
                    if is_left and (ox >= tx + tw - 0.005 and ox < c_min) and overlap_yz:
                        corridor_path.add(other.name); corridor_path.update(get_all_blockers(other, current_context, memo, visited))
                    elif not is_left and (ox + ow <= tx + 0.005 and ox + ow > c_max) and overlap_yz:
                        corridor_path.add(other.name); corridor_path.update(get_all_blockers(other, current_context, memo, visited))
                can_use_corr = True

        horizontal = corridor_path if (can_use_corr and len(corridor_path) < len(rear_path)) else rear_path
        total = top_blockers.union(horizontal)
        memo[target.name] = total
        visited.remove(target.name)
        return total

    for cp in priorities:
        remaining_now = [i for i in items if int(i.priority) <= cp]
        targets_now = [i for i in remaining_now if int(i.priority) == cp]
        memo_at_stop = {}
        for T in targets_now:
            unloading_results[T.name] = len(get_all_blockers(T, remaining_now, memo_at_stop, set()))
    return unloading_results
```

Il suddetto algoritmo esegue le seguenti operazioni:

1. Inizializzazione e calcolo dei confini geometrici

   - Mappatura del corridoio: la funzione acquisisce le dimensioni reali del mezzo di trasporto e calcola i limiti del corridoio (coordinate minima e massima lungo l'asse X) in base alla percentuale di larghezza impostata.
   - Ordinamento delle priorità: estrae tutte le priorità associate ai pacchi all'interno del furgone, considera una sola volta le priorità comuni a più pacchi e le ordina in modo decrescente; l'estrazione serve a scandire l'ordine cronologico delle fermate del corriere.

2. Simulazione dinamica del processo di scarico

   - L'algoritmo simula la situazione di carico in ogni singola fermata di consegna, alla quale corrisponde una certa Priorità.
   - Per ogni livello di priorità P (partendo dalla ultima consegna a ritroso fino alla prima), l'algoritmo crea un "contesto attuale" composto solo dai pacchi con priorità $\leq P$
   - In questo modo si assume correttamente che i pacchi con priorità più alta (destinati alle fermate precedenti) siano già stati scaricati e quindi non costituiscano più un ostacolo fisico per l'estrazione degli oggetti correnti.

3. Ricerca degli ostacoli (Funzione `get_all_blockers()`)

   Per ogni pacco target analizzato in una specifica fermata, l'algoritmo calcola l'insieme (set) di tutti gli altri pacchi che ne impediscono l'estrazione fisica; per fare ciò, l'algoritmo analizza tre direzioni di movimento:

   - Ostruzioni superiori (Asse Y - Altezza): identifica tutti i pacchi appoggiati direttamente sopra l'oggetto target (o sopra gli oggetti che a loro volta lo sovrastano); questi ostacoli devono essere rimossi per poter sollevare o sfilare l'oggetto.
   - Ostruzioni posteriori (Asse Z - Profondità): identifica tutti i pacchi che si trovano tra l'oggetto target e il portellone posteriore del furgone e che ne intersecano la sagoma; detta proiezione rappresenterebbe la via di estrazione in assenza del corridoio di accesso.
   - Ostruzioni laterali verso il corridoio (Asse X - Larghezza): se il vincolo del corridoio è attivo, l'algoritmo calcola quanti e quali pacchi si frappongono tra l'oggetto e lo spazio vuoto del corridoio centrale.

4. Ottimizzazione del percorso di estrazione

   - Una volta calcolati sia l'insieme degli ostacoli verso il portellone posteriore, sia l'insieme degli ostacoli verso il corridoio, l'algoritmo li confronta.

- Viene effettuata una scelta "greedy" a livello locale: l'algoritmo sceglie il percorso che richiede il minor numero di movimentazione di pacchi (la cardinalità minore del set di bloccanti).
- Al fine di determinare l'elenco finale degli oggetti da spostare, l'insieme di pacchi da spostare (`horizontal`) viene sommato a quello dei pacchi superiori (`top_blockers`), i quali vanno rimossi in ogni caso.

5. Gestione delle performance e dei loop

   - Memorizzazione: data la natura pseudo-ricorsiva dell'algoritmo, esso salva i risultati già computati in un dizionario di memoria (`memo`), onde evitare di ricalcolare più volte l'albero degli ostacoli che includono lo stesso pacco all'interno della stessa fermata; la memorizzazione abbatte drasticamente i tempi di calcolo ed evita errori difficili da scovare.
   - Risoluzione delle dipendenze circolari: utilizza un insieme di nodi visitati (`visited`) per evitare che l'algoritmo entri in un loop infinito qualora due pacchi si bloccassero a vicenda (situazione teoricamente possibile in geometrie complesse).

6. Output della metrica

   Infine, per ogni pacco presente nel furgone, la funzione conta la cardinalità dell'insieme finale degli ostacoli e restituisce un dizionario che mappa ogni ID pacco al numero esatto di mosse di scarico necessarie per prelevarlo.

Grazie a questo calcolo, all'interno del simulatore visuale (`test_slider.py`), i pacchi non raggiungibili direttamente vengono evidenziati con i bordi illuminati di rosso. Come si evidenzierà nel capitolo successivo, questa metrica ha permesso di dimostrare numericamente che, a parità di oggetti caricati, la presenza del corridoio riduce drasticamente il numero di mosse totali necessarie per completare l'intero giro di consegne.

4.4 Persistenza dei dati e generazione dei report (Excel)

Per consentire un'analisi statistica approfondita e ripetibile dei test effettuati, lo script `test_slider.py` implementa un sistema di persistenza dei dati basato su file in formato .csv.

Ogni volta che viene eseguita una simulazione di carico (sia singola che tramite la batteria di test automatizzata in Bash) l'algoritmo estrae le metriche chiave e le appende al file di log. Nello specifico, per ogni configurazione vengono registrati:

- il numero di oggetti totali tentati e quelli effettivamente caricati;

- la percentuale di larghezza del corridoio applicata;
- il volume totale occupato e la massa complessiva del carico;
- il numero totale di mosse operative necessarie per lo scarico del furgone (calcolato tramite l'algoritmo `calculate_moves_v5`).
- la distanza del Centro di Gravità (CoG) rispetto al baricentro del vano di carico.

Questo "database" ha consentito di conservare i risultati di centinaia di esecuzioni e di fungere da sorgente dati per la generazione automatica dei grafici di sintesi.

4.5 Generazione dei grafici di sintesi in HTML

Per studiare l'impatto del corridoio di accesso sulle performance di carico e di scarico, i dati accumulati nel file .csv vengono elaborati matematicamente e proiettati su grafici bidimensionali interattivi.

Sfruttando la libreria `Plotly`, il sistema genera automaticamente una pagina HTML (denominata ad esempio `grafici_items_60.html`) contenente tre *subplot* allineati verticalmente:

1. Grafico della capacità di carico: il grafico mostra sull'asse Y il numero di pacchi caricati al variare della larghezza del corridoio (asse X).
2. Grafico dell'efficienza operativa: mostra l'andamento del numero totale di mosse necessarie per svuotare il furgone.
3. Grafico del centro di gravità: evidenzia lo spostamento (in millimetri) del baricentro del carico, rispetto al centro perfetto del vano.

La scelta di esportare questi grafici in formato HTML (anziché come semplici immagini statiche PNG o JPG) offre il vantaggio dell'interattività: l'utente può passare il mouse sui punti del grafico per leggere i valori esatti (*hovering*), effettuare zoom su specifiche aree di interesse e isolare serie storiche di dati.

4.6 Studio dell'impatto del corridoio e scalabilità

Le simulazioni avevano dimostrato che l'inserimento di un corridoio di accesso migliorava notevolmente l'efficienza operativa del carico. A questo punto si è reso necessario condurre uno studio parametrico per comprendere come il vincolo del corridoio influisse sulle prestazioni di carico e scarico del veicolo.

Sfruttando lo script di automazione `runner.sh`, sono state eseguite simulazioni incrociate variando il numero di oggetti totali (con batterie di test da 10 e 100 pacchi) e, per ogni numero di colli, variando la dimensione del corridoio (testata dal 10% fino al 100% della larghezza del furgone).

```
    #!/bin/bash

# Nome del file in cui verranno salvati tutti i log
OUTPUT_FILE="report_simulazioni.txt"

# Svuota il file di output se esiste gi  , o lo crea se non c'
> "$OUTPUT_FILE"

echo "=== AVVIO BATTERIA DI TEST ===" | tee -a "$OUTPUT_FILE"
echo "I risultati dettagliati verranno salvati in: $OUTPUT_FILE"
echo "-----------------------------------------" >> "$OUTPUT_FILE"

# Ciclo per il numero di oggetti: da 0 a 100 con passo 10
for n in $(seq 10 10 100); do
    # Ciclo per lo spessore del corridoio: da 0.0 a 0.50 con passo 0.05
    for w in $(seq 0.1 0.1 0.9); do

        echo "Lancio test con: Pacchi = $n | Corridoio = $w" | tee -a "$OUTPUT_FILE"
        echo "-----------------------------------------" >> "$OUTPUT_FILE"

        # Eseguiamo lo script Python passando i parametri
        # Usiamo '2>&1' per catturare nel file di testo sia l'output normale che eventuali errori
        python3 test_slider.py -n "$n" -w "$w" >> "$OUTPUT_FILE" 2>&1

        # Aggiungiamo un paio di righe vuote di stacco per rendere il file leggibile
        echo -e "\n\n" >> "$OUTPUT_FILE"

    done
done

echo "=== BATTERIA DI TEST COMPLETATA ===" | tee -a "$OUTPUT_FILE"
```

4.7 Simulazione 3D e animazione dello scarico

Infine è stata prevista un'interfaccia utente offerta dalla pagina HTML dedicata alla visualizzazione tridimensionale del carico.

Terminato il calcolo euristico di posizionamento dei pacchi, l'algoritmo crea un file

HTML contenente un render 3D interattivo del vano del furgone. Questa pagina presenta due caratteristiche innovative utili per l'analisi logistica:

- la simulazione dinamica tramite Slider: nella pagina HTML viene inserito un elemento di controllo a scorrimento (`slider`). Muovendo lo slider, l'utente può simulare visivamente lo scaricamento del camion fermata dopo fermata. Gli oggetti spariscono dal furgone rispettando l'ordine dettato dalla loro priorità di consegna.

- l'evidenziazione dei pacchi bloccati: durante l'animazione generata dallo slider, l'algoritmo interroga ad ogni "fermata" la funzione di calcolo delle mosse; se ad una certa fermata un pacco deve essere scaricato e risulta fisicamente ostruito da altri oggetti, esso non viene semplicemente rimosso, ma i suoi bordi si illuminano di rosso, con ciò evidenziando i pacchi non accessibili direttamente.

Questo feedback rende l'analisi del problema non solo numerica ma anche pratica e visiva: esso permette di comprendere immediatamente se l'algoritmo ha generato un piano di carico realmente efficiente per l'operatore umano o se, invece, ha creato dei colli di bottiglia operativi.

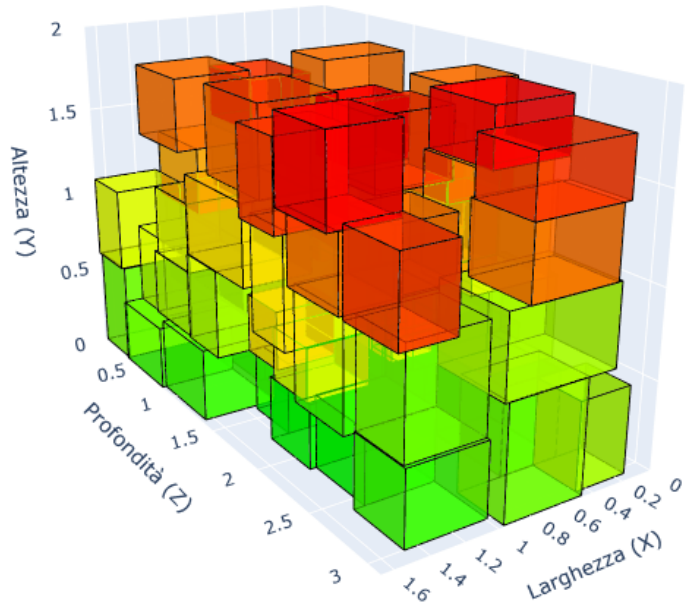

Figura 4.1: Caricamento senza corridoio

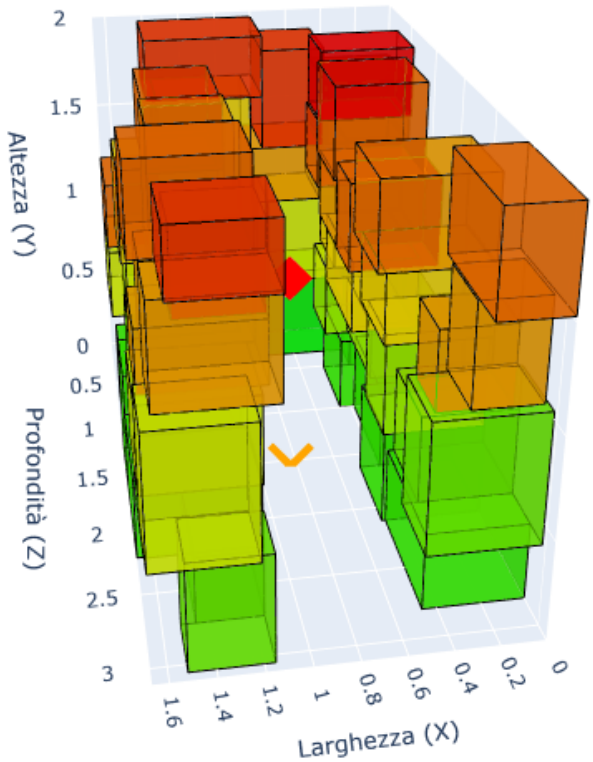

Figura 4.2: Caricamento con corridoio

Capitolo 5

Analisi dei risultati e conclusioni

5.1 Configurazione dello Stress Test

Per validare l'efficacia delle modifiche algoritmiche apportate alla libreria e studiare l'impatto del corridoio di accesso è stata condotta una consistente batteria di simulazioni. Sfruttando l'automazione dello script `runner.sh`, sono stati eseguiti test incrociati variando:

- il numero di oggetti totali generati (da batterie minime di 10 fino a stress test da 100 pacchi).
- la larghezza del corridoio centrale, testata dal 10% fino al 90% della larghezza totale del vano di carico.

I dati raccolti ed esportati nel report globale sono stati aggregati per calcolare le medie prestazionali. Di seguito vengono analizzati i tre parametri fondamentali osservati.

5.2 Analisi delle metriche sperimentali

5.2.1 Impatto sulla capacità di carico (Saturazione)

La prima metrica analizzata riguarda la perdita di capacità volumetrica del furgone in dipendenza della larghezza del corridoio.

- Corridoio stretto (10% - 40%): laddove il furgone senza corridoio riesce a stivare una media di circa 46-50 pacchi, l'introduzione di un corridoio largo fino al 40% del vano comporta una minima riduzione dei pacchi caricati e rimane a bordo una media compresa tra i 42 e i 44 pacchi; quindi, in questo range l'introduzione del corridoio non determina inefficienze di carico.

- Corridoio medio (50%): a questa percentuale inizia ad emergere un apprezzabile riduzione dei pacchi caricati, in quanto la media scende a circa 37 pacchi.

- Corridoio ampio (> 50%): in questo range i test mostrano un collasso della saturazione: il numero di pacchi crolla a una media di appena 12-14 oggetti. Infatti, il corridoio diventa talmente ampio da frammentare l'area di carico in due strisce laterali troppo strette per accogliere la maggior parte delle geometrie dei pacchi.

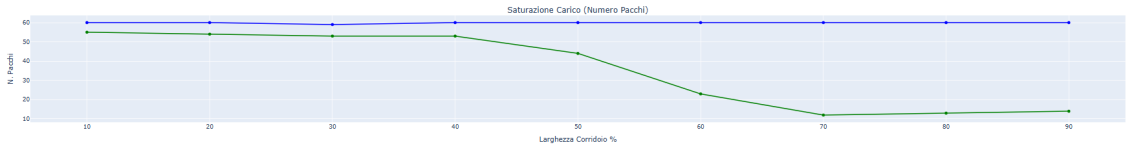

Figura 5.1: Saturazione volumetrica.

5.2.2 Impatto sull'efficienza operativa

L'efficienza operativa rappresenta il vero successo dell'algoritmo proposto, misurando quante operazioni di spostamento deve fare il corriere per estrarre il carico.

- Configurazione standard (senza corridoio): senza l'ausilio del corridoio, l'estrazione sequenziale dei pacchi richiede un numero di mosse elevatissimo, con una media oscillante tra le 174 e le 239 movimentazione totali per lo svuotamento del mezzo.

- Configurazione con corridoio attivo: l'introduzione del corridoio, anche nella sua misura minima del 10%, abbatte sensibilmente il numero delle mosse portandolo a una media di appena 10.3 movimentazioni. Allargando il corridoio (al 20% e al 30%), le mosse scendono rispettivamente a 4.8 e 3.2 movimentazioni. Oltre il 30%, il miglioramento delle mosse diventa marginale avvicinandosi allo zero assoluto (anche se, sopra il 60%, tale risultato si scontra con l'esigua quantità di pacchi effettivamente caricati nel vano).

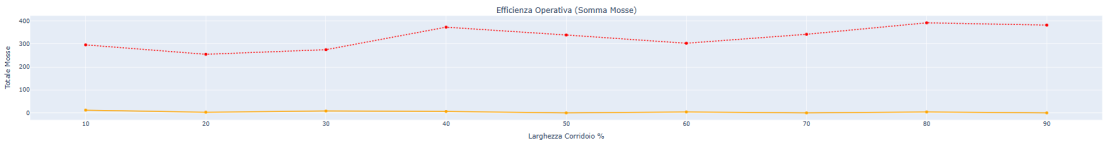

Figura 5.2: Efficienza operativa.

5.2.3 Analisi della stabilità (distanza dal Centro di Gravità)

La sicurezza stradale del furgone impone che il carico rimanga il più centrato possibile. La distanza del Centro di Gravità (CoG) reale rispetto a quello geometrico ideale è stata misurata in millimetri.

La presenza di un corridoio impone la collocazione dei pacchi sulle pareti laterali e ciò potrebbe determinare uno sbilanciamento nel caso di rilevanti asimmetrie di peso.

Di seguito vengono esposti i risultati dei test di stabilità:

- Senza corridoio: l'algoritmo classico posiziona i carichi in modo forzatamente centrato, registrando uno scostamento medio molto contenuto, tra i 60 mm e gli 80 mm.
- Con corridoio (10% - 50%): lo scostamento rimane ampiamente tollerabile per la meccanica di un veicolo commerciale, oscillando tra i 148 mm e i 255 mm.
- Con corridoio ($>$ 50%): superata la soglia critica del 50%, lo scostamento del baricentro subisce un'impennata considerevole, schizzando a oltre 730 mm (fino a superare i 1360 mm al 70%). Tale risultato è coerente con la considerazione secondo cui, potendo caricare pochissimi oggetti, questi finiscono per accumularsi tutti sul fondo.

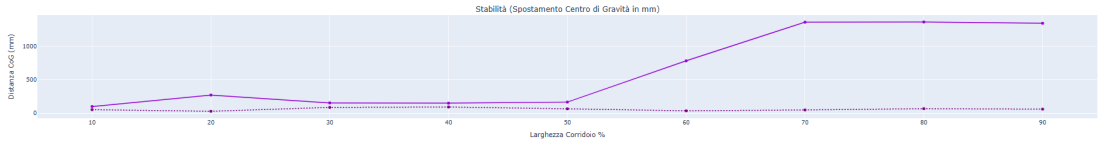

Figura 5.3: Distanza del CoG dal punto di ideale.

5.3 Conclusioni e identificazione del punto di ottimo

L'obiettivo primario di questa tesi era dimostrare che l'introduzione di un vincolo di corridoio unito all'ordinamento per priorità potesse migliorare considerevolmente l'operatività dello scarico merci, senza compromettere significativamente la capacità di trasporto del veicolo.

I dati raccolti confermano l'ipotesi.

Dall'incrocio delle tre metriche emerge chiaramente l'esistenza di un punto di ottimo che inquadra la larghezza del corridoio tra il 20% e il 40% della larghezza del vano di carico (in questo studio è stato usato il vano di un FIat Ducato L3H2).

In questo specifico intervallo (20%-40%) l'operatore logistico ottiene il massimo compromesso possibile:

- Si preserva un'ottima capacità di carico (perdendo meno del 15% dei pacchi rispetto al furgone totalmente stipato).
- Si abbatte del **98%** il lavoro fisico e il tempo di ricerca dell'operatore (portando le movimentazioni dei pacchi da oltre 200 a meno di 5).
- Si mantiene il baricentro del veicolo entro limiti di scostamento sicuri e perfettamente gestibili dai sistemi di bordo

Per apprezzare visivamente i suddetti risultati, di seguito è proposta una sovrapposizione dei dati provenienti dallo stress test. Risulta netta la tendenza al distacco successiva ad una larghezza del corridoio di accesso del 50% rispetto alla larghezza del vano di carico.

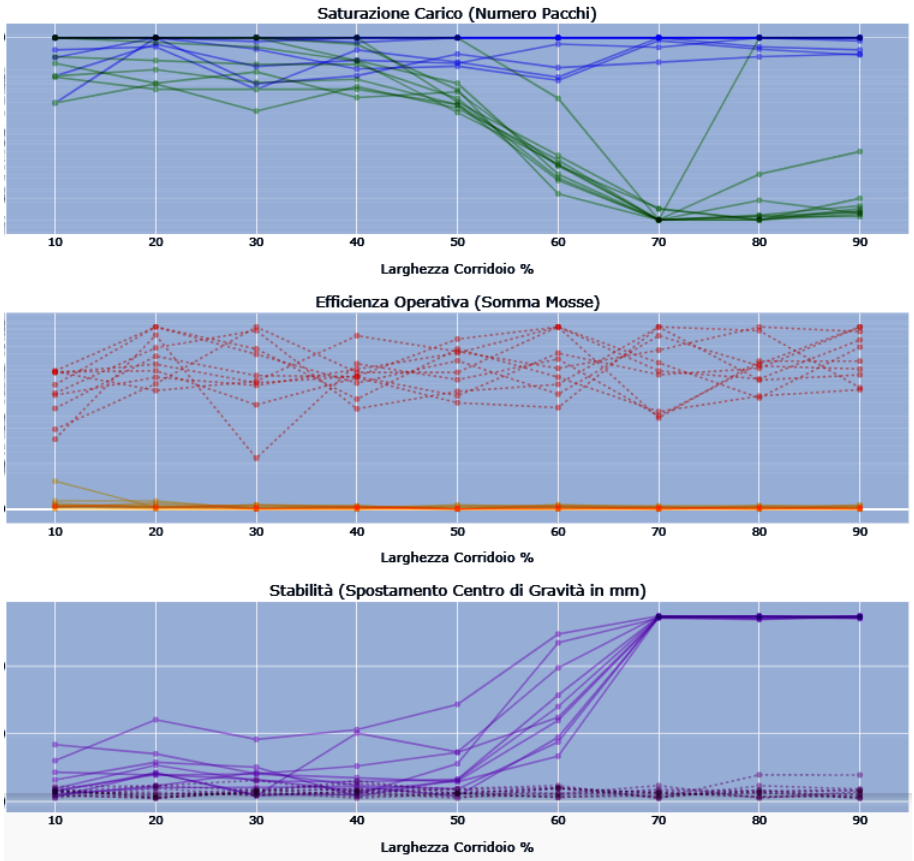

Figura 5.4: Sovrapposizione fra i grafici.

5.4 Obiettivi futuri

Durante la progettazione e la realizzazione di questa tesi sono state evidenziate nuove prospettive di lavoro.

- Modifica del motore di caricamento: con l'introduzione del corridoio, è buona norma dare più peso alle ancore lontane dallo stesso. In tal modo, nel caso di più file di pacchi fra corridoio e portellone laterale, gli ultimi colli caricati (più prioritari), risulteranno adiacenti al furgone.

- Prevedere un portellone laterale: infatti, la maggior parte dei furgoni adibiti alle consegne dell'ultimo miglio presenta un portellone laterale e tale implementazione determinerebbe, con tutta probabilità, un ulteriore miglioramento dell'efficienza operativa.

- Analisi del CoG con generazione di pacchi asimmetrici: questo studio non ha preso in considerazione l'eventualità di un sottoinsieme di pacchi molto più pesanti, voluminosi o fragili. Tale scenario è molto importante per una massima verosimiglianza dello studio rispetto alla realtà delle consegne dell'ultimo miglio.

Bibliografia

[1] Henriette Koch Corinna Krebs, Jan Fabian Ehmke. Advanced loading constraints for 3d vehicle routing problems. *Springer Nature Link*, 2021.